

Lab 2.2 — Arithmetic Operators

Objective

By the end of this file, you will be able to use all seven Python arithmetic operators (+, -, *, /, //, %, **), understand the difference between true division and floor division, and apply the modulo operator to solve real programming problems.

Background

What are Operators?

An **operator** is a symbol that tells Python to perform a specific computation on one or more values (called **operands**). Arithmetic operators work on numeric types (`int` and `float`) and produce a result.

Why This Matters in IT & Automation

Arithmetic is everywhere in programming — calculating network bandwidth, computing tax amounts, determining loop iterations, hashing algorithms, and scheduling time windows. The % (modulo) operator in particular is used constantly: checking if a number is even/odd, cycling through list indices, and more.

Real-World Applications

Operator	Real-World Use
+ - * /	Any financial calculation, sensor reading, measurement
// (floor division)	Calculating how many full pages fit in a document
% (modulo)	Checking if a value is even (<code>n % 2 == 0</code>), wrapping counters
** (exponentiation)	Compound interest, pixel scaling, cryptography

Lab Overview

Objectives	Use all 7 arithmetic operators; understand integer vs float division
Estimated Time	15–20 minutes
Prerequisites	Lab 2.1 complete — comfortable with variable assignment

Original Material

Python supports standard math operations plus a few extras that are especially useful in programming.

Code

```
a = 17
b = 5

print("Addition:      ", a + b)      # 22
print("Subtraction:   ", a - b)      # 12
print("Multiplication:", a * b)      # 85
print("Division:      ", a / b)      # 3.4 (always float)
print("Floor Division:", a // b)     # 3 (rounds DOWN)
print("Modulo:        ", a % b)      # 2 (remainder)
print("Exponentiation:", a ** 2)     # 289 (17 squared)
```

Expected Output

```
Addition:      22
Subtraction:    12
Multiplication: 85
Division:       3.4
Floor Division: 3
Modulo:         2
Exponentiation: 289
```

Code Execution

1. `a = 17` and `b = 5` store two integer values in memory.
 2. Each `print()` call passes two arguments separated by a comma: a label string and an arithmetic expression.
 3. Python evaluates the expression first (e.g., `a + b` $\rightarrow$ 22) and then passes the result to `print()`.
 4. The `print()` function automatically inserts a space between multiple arguments, which is why "Addition: " and 22 appear on the same line separated by a space.
 5. Note that `/` always returns a **float** (3.4), while `//` truncates toward negative infinity and returns an **int** when both operands are **int** (3).
-

Step-by-Step Explanation

Line	Expression	Result	Notes
a + b	17 + 5	22	Standard addition
a - b	17 - 5	12	Standard subtraction
a * b	17 * 5	85	Standard multiplication
a / b	17 / 5	3.4	True division — always returns <code>float</code>
a // b	17 // 5	3	Floor division — discards remainder, returns <code>int</code>
a % b	17 % 5	2	Modulo — remainder after floor division (17 = 3×5 + 2)
a ** 2	17 ** 2	289	Exponentiation — 17 raised to the power of 2

Key Concepts

Concept	Definition
Arithmetic operator	Symbol that performs a mathematical operation on values
True division /	Always returns a <code>float</code> , even if both operands are integers
Floor division //	Divides and rounds down (toward negative infinity)
Modulo %	Returns the remainder of floor division
Exponentiation **	Raises the left operand to the power of the right operand
Operand	The value on which an operator acts
Operator precedence	Python follows BODMAS/PEMDAS: ** → * / // % → + -

Understanding Floor Division vs True Division

```
# Floor division always truncates toward negative infinity
print(17 // 5)    # 3    (positive case: same as truncating decimal)
print(-17 // 5)   # -4   (negative case: rounds DOWN, not toward zero!)

# True division always returns float
```

```
print(10 / 2)      # 5.0 (NOT 5 – always a float)
print(10 // 2)     # 5   (integer result)
```

Understanding Modulo

The modulo operator % gives the **remainder** after division. A useful mental model:

```
17 ÷ 5 = 3 remainder 2
      ↑
    this is 17 % 5
```

Common uses:

```
# Check if a number is even
number = 42
if number % 2 == 0:
    print("Even")

# Cycle through values 0-4 repeatedly
for i in range(20):
    print(i % 5)      # 0,1,2,3,4, 0,1,2,3,4, ...
```

Common Mistakes

WRONG — Expecting / to return an integer

```
result = 10 / 2
print(result)      # Prints 5.0, NOT 5
print(type(result)) # <class 'float'>
```

CORRECT — Use // if you need an integer result

```
result = 10 // 2   # 5 (int)
```

WRONG — Confusing // rounding direction with negative numbers

```
print(-7 // 2)    # Beginners expect -3, Python gives -4 (floor toward -∞)
```

CORRECT — Know that // rounds toward negative infinity

```
print(-7 // 2)    # -4
print(int(-7/2))  # -3 (truncation toward zero – different behaviour!)
```

WRONG — Using ^ for exponentiation (that's bitwise XOR in Python)

```
print(2 ^ 3)      # 1 (XOR, not power!)
```

CORRECT — Use **

```
print(2 ** 3)     # 8
```

WRONG — Division by zero (causes `ZeroDivisionError`)

```
print(10 / 0)  # ZeroDivisionError: division by zero
```

CORRECT — Check the divisor first

```
if b != 0:  
    print(a / b)
```

Cross References

- **Previous concept:** [Lab_2_1_Variables_and_DataTypes.md] — Variables and types
- **Next concept:** [Lab_2_3_StringOps_TypeConversion.md] — String operations and type conversion
- **Practice:** [Practice_Problems.md] — Problems 1 (temperature formula), 3 (paycheck), 4 (comparisons)
- **Quick reference:** [Revision_Sheet.md] — Key Syntax cheat sheet